Catching Different (Error) Exceptions

We need to take a moment to discuss another object-oriented programming (OOP) fundamental: *inheritance*. Classes – as we've seen – serve as templates for objects; you make objects using the `new` keyword and a class name. It is also possible to define a class using another as a base. The keywords `class` and `extends` are used to tell PHP both that a new class exists and that it uses another class as its base (or parent). The statement `class MyException extends Exception` means that PHP creates a new class `MyException`, which has all the features of `Exception`. In OOP terms this is inheritance; `MyException` is a *subclass* (or *child*) of the parent `Exception` class; it will inherit all of the properties and methods of the parent and will behave exactly like it. Child classes can introduce new functionality or override parent functionality (to an extent), but this is beyond the scope of this chapter and will be covered in Chapter 6, "Object Fundamentals." Inheritance can be chained together: a base class `A` can have a child `B`, and `B` can have a child `C`, creating a (tree) hierarchy of relationships between them.

PHP's `Error` class is the parent of several child classes, such as `ArithmeticError`, `TypeError`, and `ValueError`. In turn, some of those have their own children: `ArithmeticError` has a child `DivisionByZeroError`, and `TypeError` has a child, `ArgumentCountError`. In this way, exceptions can be specialized, which is good because throwing and handling `Exception` objects everywhere is going to be tedious: you'd need to wade through all sorts of messages or other properties of each object to determine what the precise problem is. By introducing a hierarchy of exception classes, it becomes possible to elegantly switch paths in your code based on the exception's class, because you can attach as many `catch` blocks as you like to a `try`, using the type declaration of the object argument to filter the exceptions that are allowed into that block. Conceptually, the class used to create the exception object becomes part of the message, meaning you can handle `DivisionByZeroError` exceptions using a different code block to `TypeError` exceptions:

```
function surpriseBoom()
{
    if (rand(1, 10) % 2 === 0) {
        notDefined();
    } else {
        print 1 / 0;
    }
}
```